6

LECTURE 6

Tree Traversals, Iterators, Templates

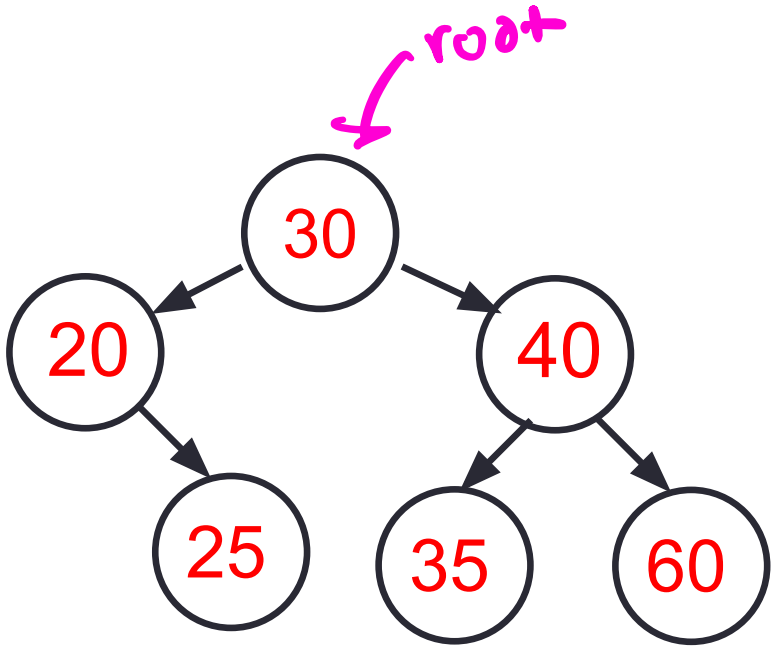
CS24 · Fall 2026 · Diba Mirza

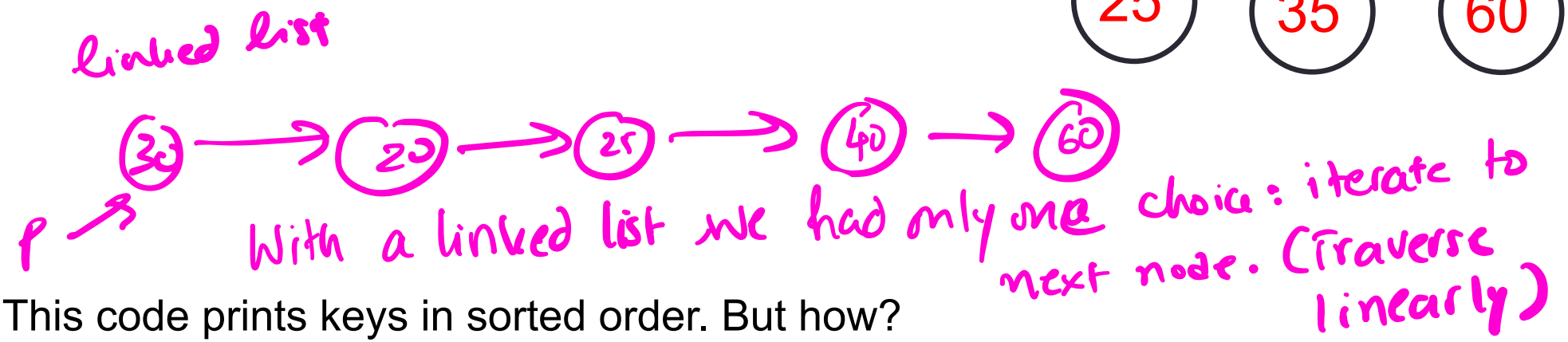
https://join.iclicker.com/BCRL

Traversing std::set

```
std::set<int> s = {30, 20, 25, 40,35, 60};

for (int x : s) {
    cout << x << " ";
}
```

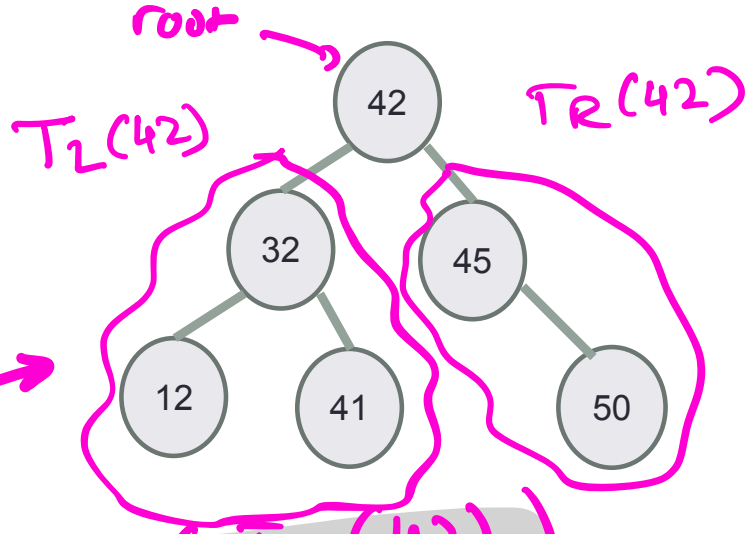




This code prints keys in sorted order. But how?

Inorder traversal: use to visit nodes in a BST in sorted order

```
inorder(r: pointer to the root node):
    if r is null: return
    inorder(r->left)
    process r (e.g., print r->val)
    inorder(r->right)
```



Trace inorder on this bst

inorder $T_L(42)$, 42 , inorder $T_R(42)$

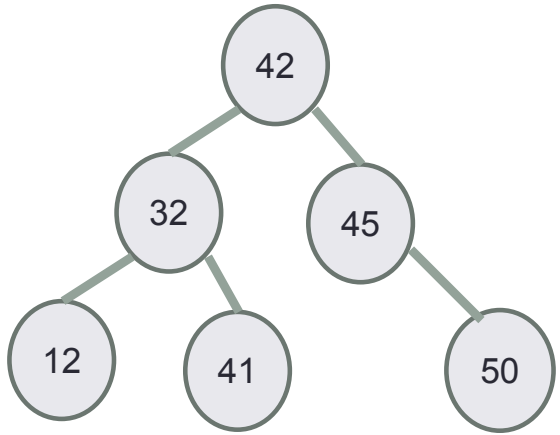
inorder $(T_L(32))$, 32, inorder $T_R(32)$, 42, 42, 45, 50

Final output: 12, 32, 41 , 42, 45, 50

$T_L(42)$ $T_R(42)$

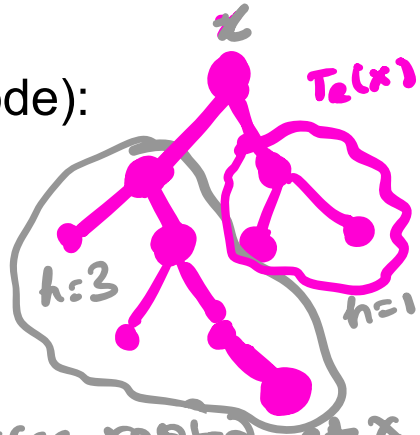
Pre-order traversal: a linear representation of the tree

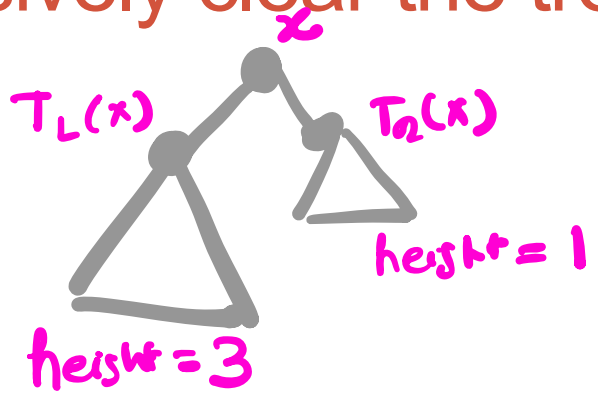
```
preorder(r : pointer to pointer to the root node):
    if r is null, return
    process r (e.g., print r->val)
    preorder(r->left)
    preorder(r->right)
```



Post-order traversal: use to recursively clear the tree!

```
postorder(r : pointer to the root node):
    if r is null, return
    postorder(r->left)
    postorder(r->right)
    process r (e.g., print r->val)
```





Why would preorder not work for clear?

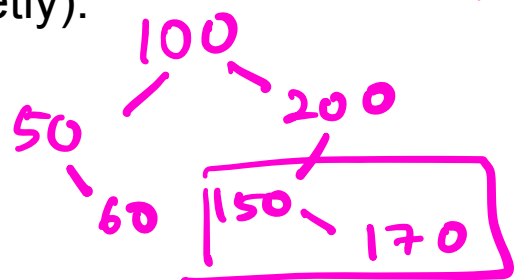
```
int bst::getHeight(Node *r) const{
    if (!r)
        return -1;
    int hleft = getHeight(r->left);
    int hright = getHeight(r->right);
    return max(hleft, hright) + 1;
}
```

// 3
// 1
max (3, 1) +1 = 4

```
void bst::clear(Node *r){
    if (!r)
        return;
    clear(r->left);
    clear(r->right);
    delete r;
}
```

Pre-order Traversal Game!

1. **Draw a BST:** Individually, draw a BST with 6 distinct keys (e.g., integers 1–10). Your Your BST (draw secretly):



100, 50, 60 — $T_L(100)$

200, 150, 170 — $T_R(100)$

2. **Trace Preorder Sequence:** Trace the preorder traversal (root, left, right) of your BST and write the sequence of node values.

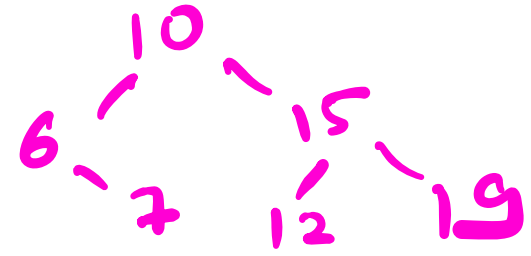
Your Preorder sequence: 100, 50, 60, 200, 150, 170

(read the sequence to your partner, don't show them your BST diagram!)

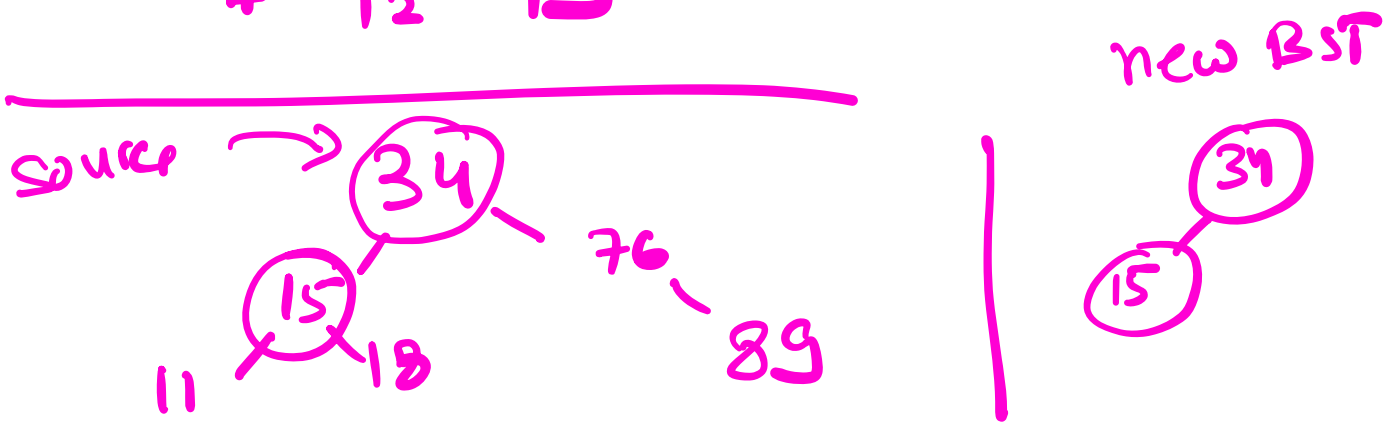
Pre-order traversal Game!

3. Write the Pre-order sequence you received:

4. Reconstruct the BST: Using your partner's sequence, rebuild their BST by inserting nodes in the given order, respecting BST properties.



5. Compare trees.



When would you use each traversal and why?

Inorder: useful when. . . visit / print in sorted order

Postorder: useful when. . . implementing destructor

Preorder: useful when. . . implementing copy constructor

Running time of all traversals pre / post / inorder : O(n)

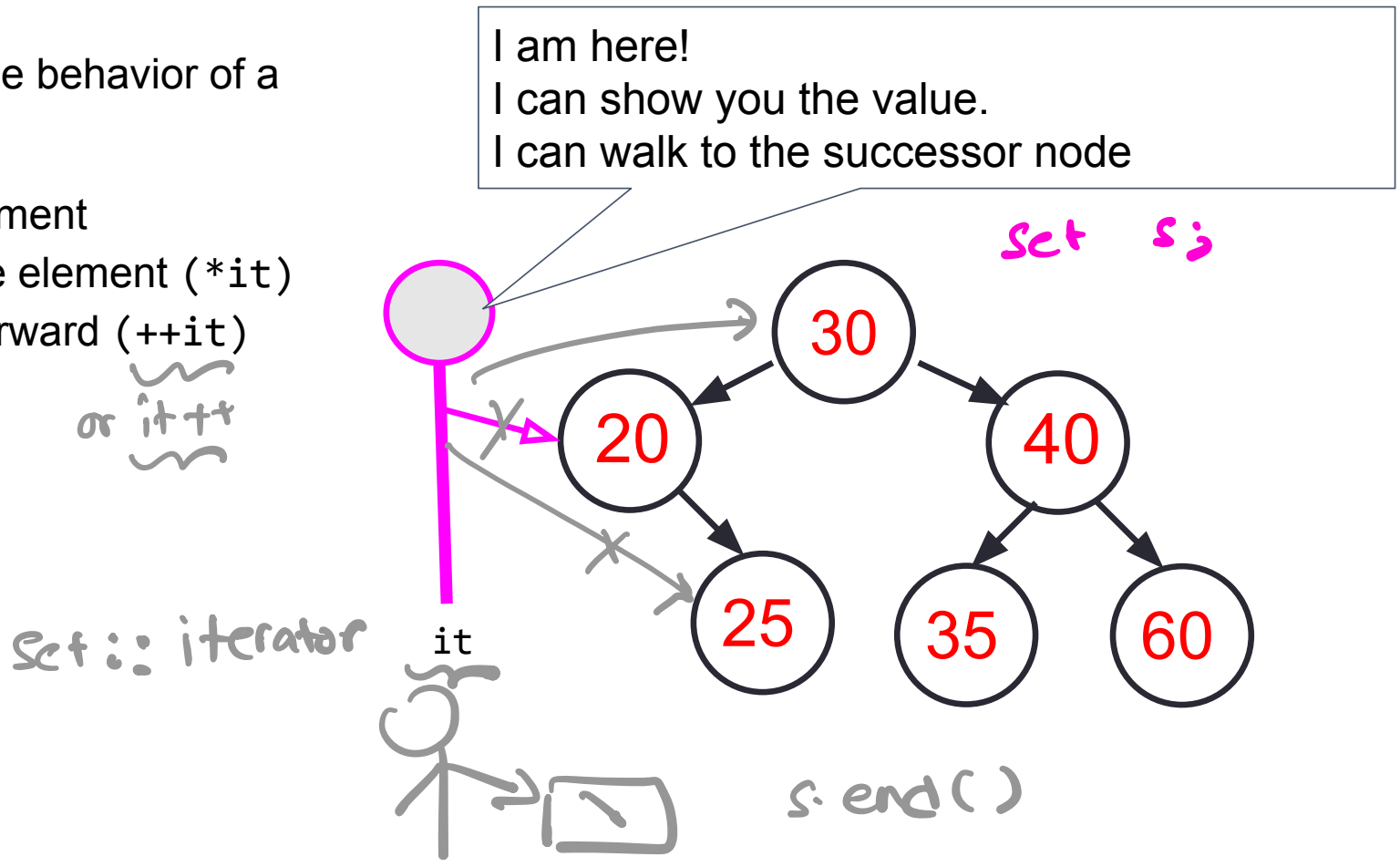
C++STL

- The C++ Standard Template Library is a handy set of three built-in components:
  - Containers: Data structures
  - Iterators: Standard way to traverse containers
  - Algorithms: These are what we ultimately use to solve problems

What is an iterator?

An iterator abstracts the behavior of a traversal pointer

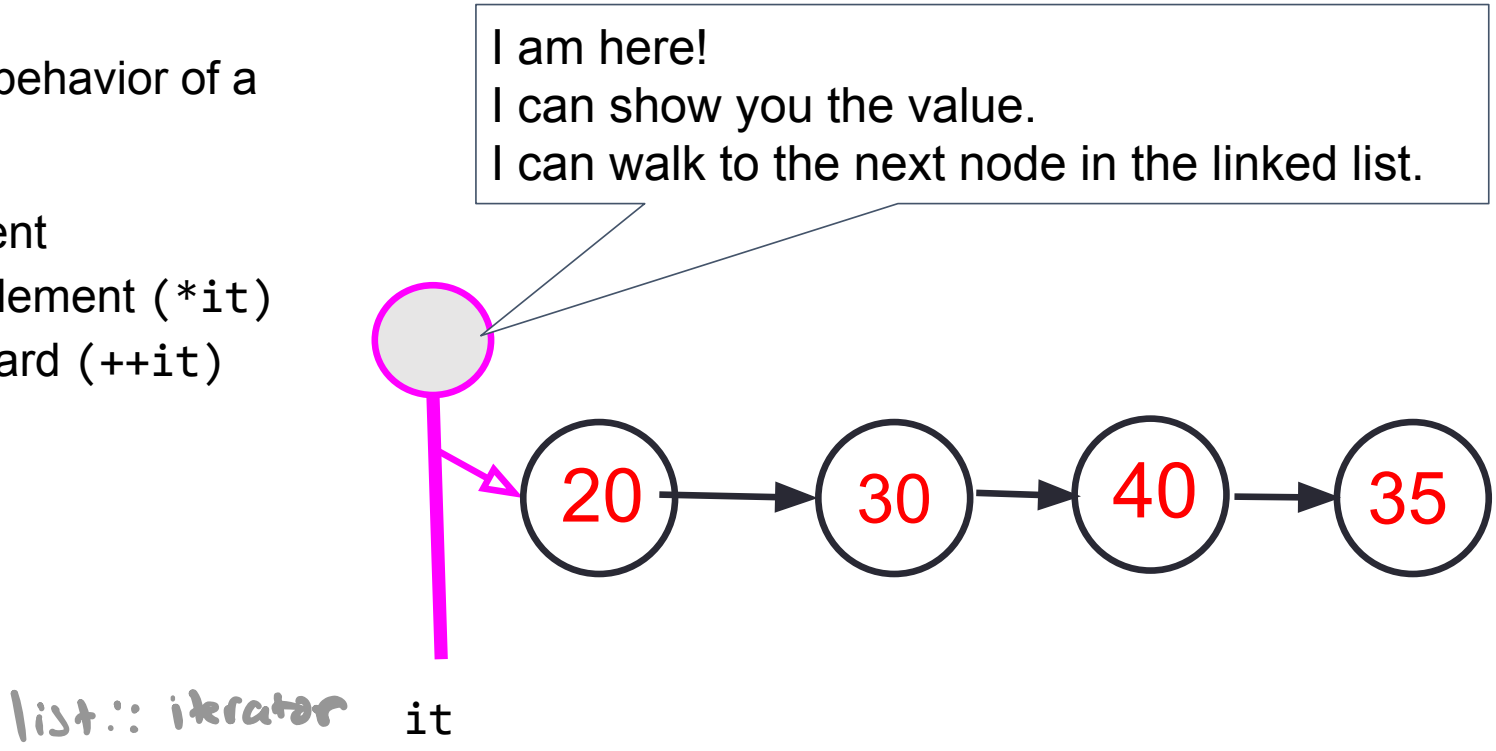
- It points at one element
- You can look at the element (`*it`)
- You can move it forward (`++it`)



What is an iterator?

An iterator abstracts the behavior of a traversal pointer

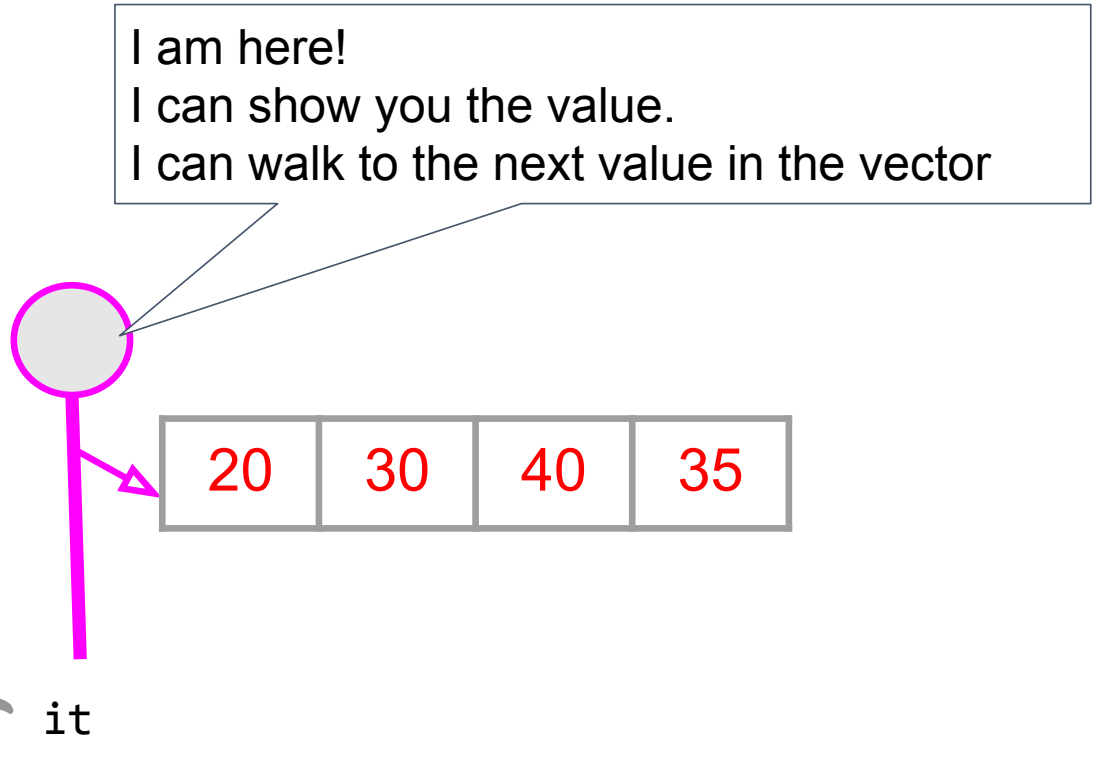
- It points at one element
- You can look at the element (`*it`)
- You can move it forward (`++it`)



What is an iterator?

An iterator abstracts the behavior of a traversal pointer

- It points at one element
- You can look at the element (`*it`)
- You can move it forward (`++it`)



Discuss (2 mins):
What functions does iterator class need to allow operations like `*it` and `++it`?

Same pattern works on different containers (set, vector, linked list…)

```
for (auto it = v.begin(); it != v.end(); ++it)  // vector

for (auto it = s.begin(); it != s.end(); ++it)  // set

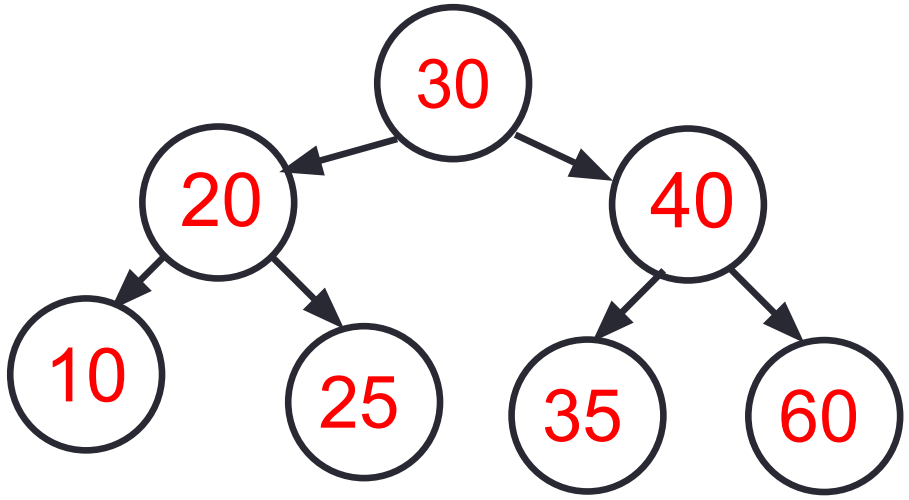
for (auto it = m.begin(); it != m.end(); ++it)  // list
```

An iterator abstracts the behavior of a traversal pointer

- It points at one element
- You can look at the element `(*it)`
- You can move it forward `(++it)`

Why not write this code in main() to iterate through the bst in sorted order?

```
Node* r = b.min(root);
while(r){
   cout << r->data << " ";
   r = b.successor(r);
}
```



A. It is slower than an iterator

B. It exposes the internal Node* and successor details

C. It only works for BSTs, not for other containers

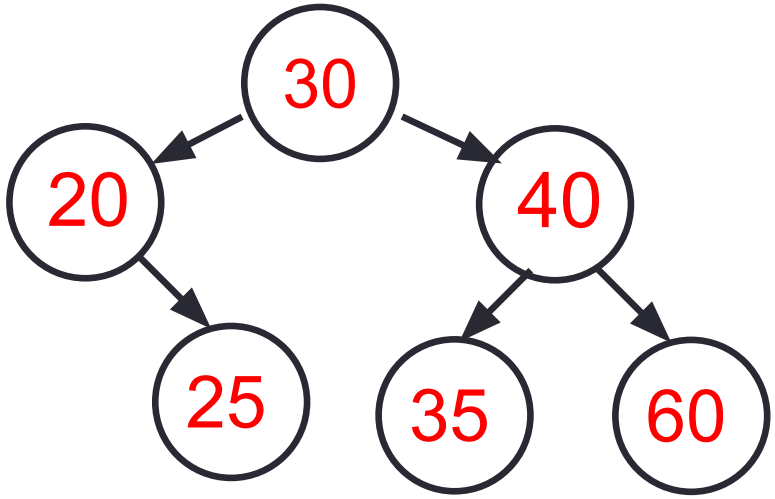
D. Both B and C

E. It is fine, no reason to avoid it

Traversing std::set

```
std::set<int> s = {30, 20, 25, 40,35, 60};

for (int x : s) {
    cout << x << " ";
}
```



This code prints keys in sorted order. But how?

One function, any type

T is a placeholder -- the compiler fills it in with whatever type you pass.

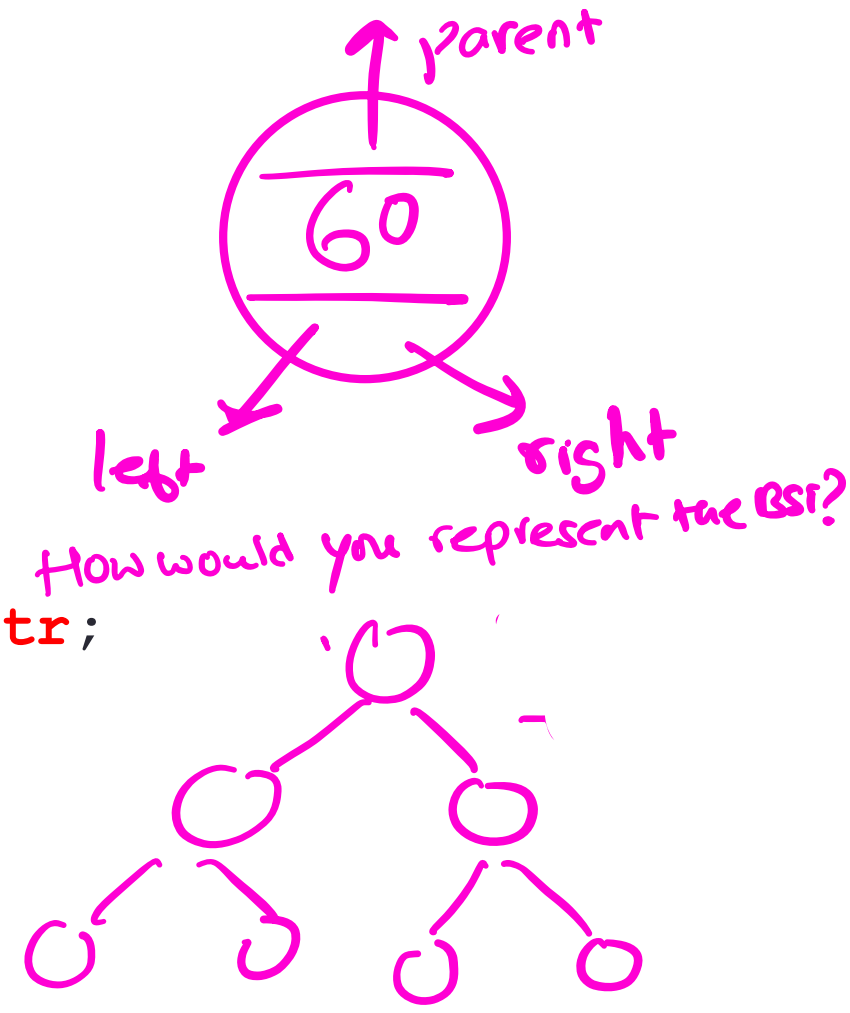
```
template <class T>
T myMax(T a, T b) {
    return (a > b) ? a : b;
}
```

myMax(3, 7) -- T is int
myMax(3.5, 2.1) -- T is double
myMax('a', 'z') -- T is char

```
struct TreeNode {

   TreeNode* left;
   TreeNode* right;
   TreeNode* parent;
   int const data;

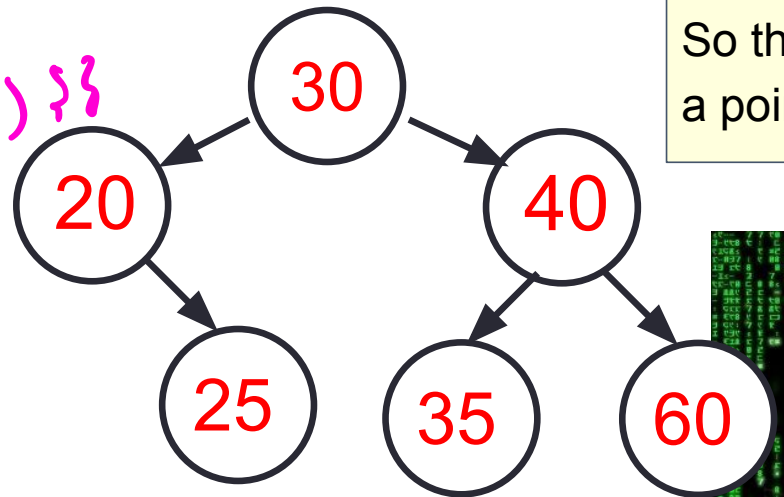
   TreeNode(int d) : data(d) {
      left = right = parent = nullptr;
   }
};
```

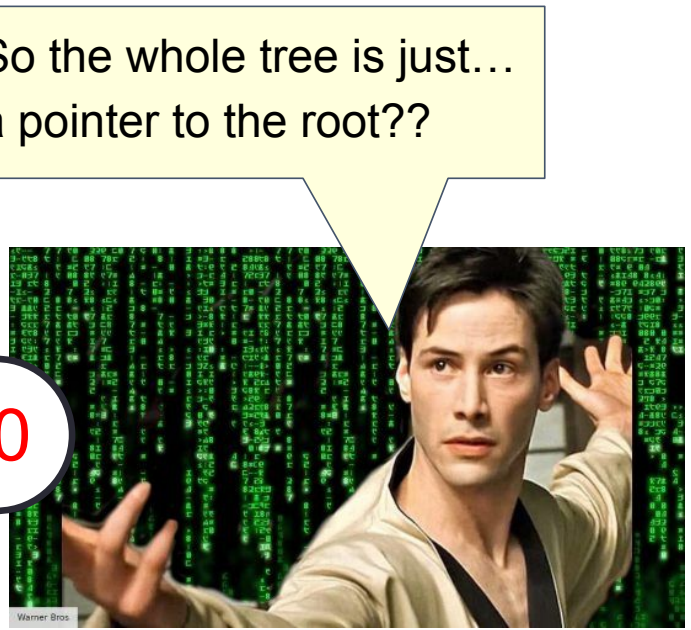


ADT Design of BST

```
class bst {
  public:
    bst(): root(nullptr) {}

  private:
    TreeNode *root;
};
```





In this week's lab you will implement a bst.

(5 mins) Write the constructor and any two public member functions

Looking ahead …

Quiz 2 topics: Same topics as quiz 1 + complexity analysis (no BSTs)

Tour of caliber (now)

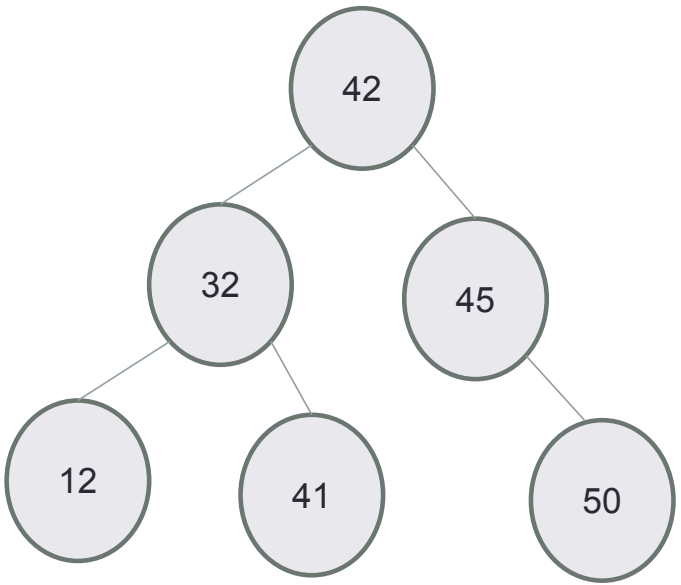
Complete by Friday:

- Schedule your mock interview on Caliber
- Make progress on lab02

Complete before quiz 2 on Caliber:

- Weekly Practice assignment 1 & 2
- HW studio week 2 assignment

Post lecture practice: Interview question: Write a function to extract BST keys into a vector. The extraction order must allow reconstructing the exact same tree structure by sequential insertion.



https://leetcode.com/problems/binary-tree-preorder-traversal/description/?envType=p